Name: Eshitha Vimalan
Registration Number: 24BCE0921
Course Code: BCSE202P
Course Title: Data Structures and Algorithms Lab
Slot: L15+ L16

Q1: Linked List

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
   int data;
   struct Node* next;
};
void insertAtBeginning(struct Node** head_ref, int new_data)
{
   struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
   new_node->data = new_data;
   new_node->next = (*head_ref);
   (*head_ref) = new_node;
}
void insertAfter(struct Node* prev_node, int new_data)
{
   if (prev_node == NULL)
   {
      printf("the given previous node cannot be NULL");
      return;
   }
   struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
   new_node->data = new_data;
   new_node->next = prev_node->next;
   prev_node->next = new_node;
}
void insertAtEnd(struct Node** head_ref, int new_data)
{
   struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
   struct Node* last = *head_ref;
   new_node->data = new_data;
   new_node->next = NULL;
   if (*head_ref == NULL)
   {
      *head_ref = new_node;
      return;
   }
while (last->next != NULL)
   last = last->next;
   last->next = new_node;
   return;
}
```

Eshitha 24BCE0921

```
void deleteNode(struct Node** head_ref, int key)
{
   struct Node *temp = *head_ref, *prev;
   if (temp != NULL && temp->data == key)
   {
      *head_ref = temp->next;
      free(temp);
      return;
   }
while (temp != NULL && temp->data != key)
{
   prev = temp;
   temp = temp->next;
}
if (temp == NULL)
return;
prev->next = temp->next;
free(temp);
}
int searchNode(struct Node** head_ref, int key)
{
   struct Node* current = *head_ref;
   while (current != NULL)
   {
      if (current->data == key)
      return 1;
      current = current->next;
   }
   return 0;
}
void sortLinkedList(struct Node** head_ref)
{
   struct Node *current = *head_ref, *index = NULL;
   int temp;
   if (head_ref == NULL)
   {
      return;
   }
   else
   {
      while (current != NULL)
   {
   index = current->next;
   while (index != NULL)
   {
      if (current->data > index->data)
   {
   temp = current->data;
   current->data = index->data;
   index->data = temp;
}
   index = index->next;
```

Eshitha 24BCE0921

```
}
    current = current->next;
}
}
}
void printList(struct Node* node)
{
    while (node != NULL)
    {
        printf(" %d ", node->data);
        node = node->next;
    }
}
int main()
{
struct Node* head = NULL;
    insertAtEnd(&head, 10);
    insertAtBeginning(&head, 20);
    insertAtBeginning(&head, 30);
    insertAtEnd(&head, 40);
    insertAfter(head->next, 50);
    printf("Linked list: ");
    printList(head);
    printf("\nAfter deleting an element: ");
    deleteNode(&head, 30);
    printList(head);
    int item_to_find = 30;
    if (searchNode(&head, item_to_find))
    {
        printf("\n%d is found", item_to_find);
    }
    else
    {
        printf("\n%d is not found", item_to_find);
    }
    sortLinkedList(&head);
    printf("\nSorted List: ");
    printList(head);
}
```

Output:

```
Linked list:  30  20  50  10  40
After deleting an element:  20  50  10  40
30 is not found
Sorted List:  10  20  40  50
Process returned 0 (0x0)   execution time : 0.021 s
Press any key to continue.
```

Esritha 24BCE0921

```
Linked list:  30  20  50  10  40
After deleting an element:  20  50  10  40
40 is found
Sorted List:  10  20  40  50
Process returned 0 (0x0)   execution time : 0.072 s
Press any key to continue.
```

Q2: Stack Using Linked List

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
      int data;
      struct Node *next;
};
struct Node *head=NULL;

void push(int val)
{
   struct Node *newnode=malloc(sizeof(struct Node));
   newnode->data=val;
   newnode->next=head;
   head=newnode;
}

void pop()
{
   struct Node *temp;
   if(head == NULL)
      printf("Stack is Empty\n");
   else
   {
      printf("Popped element = %d\n", head->data);
      temp = head;
      head = head->next;
      free(temp);
   }
}
void printList()
{
   struct Node *temp = head;
   while(temp != NULL)
   {
   printf("%d->", temp->data);
   temp = temp->next;
   }
   printf("NULL\n");
}
int main()
```

Eshitha 24BCE0921

```
{
   push(5);
   push(15);
   push(25);
   printf("Linked List\n");
   printList();
   pop();
   printf("After the pop, the new linked list\n");
   printList();
   pop();
   printf("After the pop, the new linked list\n");
   printList();
   return 0;
}
```

Output:

```
Linked List
25->15->5->NULL
Popped element = 25
After the pop, the new linked list
15->5->NULL
Popped element = 15
After the pop, the new linked list
```

Q3: Queue Using Linked List

Source Code:

```
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
 int data;
 struct Node* next;
} Node;
Node* createNode(int new_data)
{
 Node* new_node = (Node*)malloc(sizeof(Node));
 new_node->data = new_data;
 new_node->next = NULL;
 return new_node;
}
typedef struct Queue {
 Node *front, *rear;
} Queue;

Queue* createQueue()
{
 Queue* q = (Queue*)malloc(sizeof(Queue));
 q->front = q->rear = NULL;
 return q;
```

Esnitha 24BCE0921

```
}

int isEmpty(Queue* q)
{

 if (q->front == NULL && q->rear == NULL) {
 return 1;
 }
 return 0;
}

void enqueue(Queue* q, int new_data)
{

 Node* new_node = createNode(new_data);

 if (q->rear == NULL) {
 q->front = q->rear = new_node;
 return;
 }

 q->rear->next = new_node;
 q->rear = new_node;
}

void dequeue(Queue* q)
{

 if (isEmpty(q)) {
 printf("Queue Underflow\n");
 return;
 }

 Node* temp = q->front;
 q->front = q->front->next;

 if (q->front == NULL)
 q->rear = NULL;
 free(temp);
}

int getFront(Queue* q)
{

 if (isEmpty(q)) {
       printf("Queue is empty\n");
 return INT_MIN;
 }
 return q->front->data;
}

int getRear(Queue* q)
```

Esnitha 24BCE0921

```
{

 if (isEmpty(q)) {
 printf("Queue is empty\n");
 return INT_MIN;
 }
 return q->rear->data;
}

int main()
{
 Queue* q = createQueue();

 enqueue(q, 10);
 enqueue(q, 20);

 printf("Queue Front: %d\n", getFront(q));
 printf("Queue Rear: %d\n", getRear(q));

 dequeue(q);
 dequeue(q);

 enqueue(q, 30);
 enqueue(q, 40);
 enqueue(q, 50);

 dequeue(q);
 printf("After enqueueing and dequeueing: \n");
 printf("Queue Front: %d\n", getFront(q));
 printf("Queue Rear: %d\n", getRear(q));
 return 0;
}
```

Output:

```
Queue Front: 10
Queue Rear: 20
After enqueueing and dequeueing:
Queue Front: 40
Queue Rear: 50
```

Q4: Doubly Linked List

Source Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
struct node {
```

Eswitha 24BCE0921

```
    int data;
    int key;
    struct node *next;
    struct node *prev;
};
struct node *head = NULL;
struct node *last = NULL;
struct node *current = NULL;

bool isEmpty(){
    return head == NULL;
}

void displayForward(){
    struct node *ptr = head;
    printf("\n[ ");
    while(ptr != NULL) {
        printf("(%d,%d) ",ptr->key,ptr->data);
        ptr = ptr->next;
 }
 printf(" ]");
}
void displayBackward(){
    struct node *ptr = last;
    printf("\n[ ");
    while(ptr != NULL) {
        printf("(%d,%d) ",ptr->key,ptr->data);
        ptr = ptr ->prev;
    printf(" ");
 }
 printf(" ]");
}
void insertFirst(int key, int data){
    struct node *link = (struct node*) malloc(sizeof(struct node));
    link->key = key;
    link->data = data;
    if(isEmpty()) {
        last = link;
 }
 else {
    head->prev = link;
 }
    link->next = head;
    head = link;
}
void insertLast(int key, int data){
    struct node *link = (struct node*) malloc(sizeof(struct node));
    link->key = key;
    link->data = data;
    if(isEmpty()) {
        last = link;
 }
```

Eshitha 24BCE0921

```
 else {
   last->next = link;
   link->prev = last;
 }
   last = link;
}
struct node* deleteFirst(){
   struct node *tempLink = head;
   if(head->next == NULL) {
       last = NULL;
 }
 else {
   head->next->prev = NULL;
 }
 head = head->next;
 return tempLink;
}
   struct node* deleteLast(){
   struct node *tempLink = last;
   if(head->next == NULL) {
       head = NULL;
 }
 else {
   last->prev->next = NULL;
 }
 last = last->prev;
 return tempLink;
}
   struct node* delete(int key){
   struct node* current = head;
   struct node* previous = NULL;
 if(head == NULL) {
   return NULL;
 }
 while(current->key != key) {
   if(current->next == NULL) {
       return NULL;
 }
 else {
   previous = current;
   current = current->next;
 }
 }
   if(current == head) {
       head = head->next;
 }
 else {
   current->prev->next = current->next;
 }
 if(current == last) {
   last = current->prev;
 }
```

Esnitha 24BCE0921

```
 else {
    current->next->prev = current->prev;
 }
 return current;
 }
bool insertAfter(int key, int newKey, int data){
    struct node *current = head;
    if(head == NULL) {
        return false;
 }
    while(current->key != key) {
        if(current->next == NULL) {
            return false;
 }
 else {
        current = current->next;
 }
 }
    struct node *newLink = (struct node*) malloc(sizeof(struct node));
    newLink->key = key;
    newLink->data = data;
    if(current == last) {
        newLink->next = NULL;
        last = newLink;
 }
 else {
        newLink->next = current->next;
        current->next->prev = newLink;
 }
 newLink->prev = current;
 current->next = newLink;
 return true;
}
int main(){
    insertFirst(1,10);
    insertFirst(2,20);
    insertFirst(3,30);
    insertFirst(4,1);
    insertFirst(5,40);
    insertFirst(6,56);
    printf("\nList (First to Last): ");
    displayForward();
    printf("\n");
    printf("\nList (Last to first): ");
    displayBackward();
    printf("\nList , after deleting first record: ");
    deleteFirst();
    displayForward();
    printf("\nList , after deleting last record: ");
    deleteLast();
    displayForward();
    printf("\nList , insert after key(4) : ");
```

Eshitha 24BCE0921

```
    insertAfter(4,7, 13);
    displayForward();
    printf("\nList , after delete key(4) : ");
    delete(4);
    displayForward();
}
```

Output:

```
List (First to Last):
[ (6,56) (5,40) (4,1) (3,30) (2,20) (1,10)  ]

List (Last to first):
[ (1,10)  (2,20)  (3,30)  (4,1)  (5,40)  (6,56)   ]
List , after deleting first record:
[ (5,40) (4,1) (3,30) (2,20) (1,10)  ]
List , after deleting last record:
[ (5,40) (4,1) (3,30) (2,20)  ]
List , insert after key(4) :
[ (5,40) (4,1) (4,13) (3,30) (2,20)  ]
List , after delete key(4) :
[ (5,40) (4,13) (3,30) (2,20)  ]
Process returned 0 (0x0)   execution time : 0.085 s
Press any key to continue.
```

Q5: Polynomial Addition Using Linked List

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff;
    int pow;
    struct Node* next;
};

struct Node* createNode(int c, int p) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->coeff = c;
    newNode->pow = p;
    newNode->next = NULL;
    return newNode;
}

struct Node* addPolynomial(struct Node* head1, struct Node* head2) {
    struct Node* dummy = createNode(0, 0);
    struct Node* prev = dummy;
    struct Node *curr1 = head1, *curr2 = head2;

    while (curr1 != NULL && curr2 != NULL) {
```

Eshitha 24B 2021

```
        if (curr1->pow < curr2->pow) {
            prev->next = curr2;
            prev = curr2;
            curr2 = curr2->next;
        }
        else if (curr1->pow > curr2->pow) {
            prev->next = curr1;
            prev = curr1;
            curr1 = curr1->next;
        }
        else {
            curr1->coeff = curr1->coeff + curr2->coeff;
            prev->next = curr1;
            prev = curr1;
            curr1 = curr1->next;
            curr2 = curr2->next;
        }
    }

    if (curr1 != NULL)
        prev->next = curr1;

    if (curr2 != NULL)
        prev->next = curr2;

    return dummy->next;
}

void printPolynomial(struct Node* head) {
    struct Node* curr = head;
    while (curr != NULL) {
        if (curr->coeff > 0 && curr != head)
            printf(" + ");
        else if (curr->coeff < 0)
            printf(" - ");

        int absCoeff = (curr->coeff < 0) ? -curr->coeff : curr->coeff;
        printf("%d", absCoeff);

        if (curr->pow != 0)
            printf("x^%d", curr->pow);

        curr = curr->next;
    }
    printf("\n");
}

int main() {
    struct Node* head1 = createNode(5, 2);
    head1->next = createNode(4, 1);
    head1->next->next = createNode(2, 0);
```

Eshitha 24BCE0921

```
    struct Node* head2 = createNode(-5, 1);
    head2->next = createNode(-5, 0);

    printf("1st Polynomial: ");
    printPolynomial(head1);

    printf("2nd Polynomial: ");
    printPolynomial(head2);

    struct Node* head = addPolynomial(head1, head2);

    printf("Resultant Polynomial: ");
    printPolynomial(head);

    return 0;
}
```

Output:

```
1st Polynomial: 5x^2 + 4x^1 + 2
2nd Polynomial:  - 5x^1 - 5
Resultant Polynomial: 5x^2 - 1x^1 - 3

Process returned 0 (0x0)   execution time : 0.072 s
Press any key to continue.
```

Q6: Multiplication of Polynomials Using Linked List

Source Code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff, power;
    struct Node* next;
};

struct Node* addnode(struct Node* start, int coeff, int power)
{
    struct Node* newnode = (struct Node*)malloc(sizeof(struct Node));
    newnode->coeff = coeff;
    newnode->power = power;
    newnode->next = NULL;

    if (start == NULL)
        return newnode;

    struct Node* ptr = start;
```

Eshitha 2 E0921

```
    while (ptr->next != NULL)
        ptr = ptr->next;
    ptr->next = newnode;
    return start;
}

void printList(struct Node* ptr)
{
    while (ptr != NULL) {
        printf("%dx^%d", ptr->coeff, ptr->power);
        if (ptr->next != NULL && ptr->next->coeff >= 0)
            printf("+");
        ptr = ptr->next;
    }
    printf("\n");
}

void removeDuplicates(struct Node* start)
{
    struct Node *ptr1, *ptr2, *dup;
    ptr1 = start;

    while (ptr1 != NULL && ptr1->next != NULL) {
        ptr2 = ptr1;
        while (ptr2->next != NULL) {
            if (ptr1->power == ptr2->next->power) {
                ptr1->coeff = ptr1->coeff + ptr2->next->coeff;
                dup = ptr2->next;
                ptr2->next = ptr2->next->next;
                free(dup);
            }
            else
                ptr2 = ptr2->next;
        }
        ptr1 = ptr1->next;
    }
}

struct Node* multiply(struct Node* poly1, struct Node* poly2, struct Node* poly3)
{
    struct Node *ptr1, *ptr2;
    ptr1 = poly1;

    while (ptr1 != NULL) {
        ptr2 = poly2;
        while (ptr2 != NULL) {
            int coeff = ptr1->coeff * ptr2->coeff;
            int power = ptr1->power + ptr2->power;
            poly3 = addnode(poly3, coeff, power);
            ptr2 = ptr2->next;
        }
        ptr1 = ptr1->next;
```

Eshitha 24BCE0921

```
    }

    removeDuplicates(poly3);
    return poly3;
}

int main()
{
    struct Node *poly1 = NULL, *poly2 = NULL, *poly3 = NULL;

    poly1 = addnode(poly1, 3, 3);
    poly1 = addnode(poly1, 6, 1);
    poly1 = addnode(poly1, -9, 0);

    poly2 = addnode(poly2, 9, 3);
    poly2 = addnode(poly2, -8, 2);
    poly2 = addnode(poly2, 7, 1);
    poly2 = addnode(poly2, 2, 0);

    printf("1st Polynomial: ");
    printList(poly1);

    printf("2nd Polynomial: ");
    printList(poly2);

    poly3 = multiply(poly1, poly2, poly3);

    printf("Resultant Polynomial: ");
    printList(poly3);

    return 0;
}
```

Output:

```
1st Polynomial: 3x^3+6x^1-9x^0
2nd Polynomial: 9x^3-8x^2+7x^1+2x^0
Resultant Polynomial: 27x^6-24x^5+75x^4-123x^3+114x^2-51x^1-18x^0
```

Esitha 24BCE0921